

Optimizing Ocean Surface Temperature Prediction Models through Convolutional Neural Networks (CNN)

Boris Gordeev

Table of Contents

INTRODUCTION	3
BACKGROUND AND MOTIVATION	3
RESEARCH QUESTION AND OBJECTIVES	3
SIGNIFICANCE AND SCOPE	3
ROLE OF COMPUTING IN OCEAN SURFACE TEMPERATURE PREDICTION	4
RELEVANCE OF CNN TECHNIQUES	5
METHODOLOGY	5
EXPLANATION OF CNN TECHNIQUES	5
DATA COLLECTION AND PREPROCESSING	6
EXPERIMENTAL SETUP	6
LITERATURE REVIEW	9
CURRENT STATE OF OCEAN SURFACE TEMPERATURE PREDICTION MODELS	9
EVOLUTION IN RESOLUTION (1960-TODAY)	10
AI IMPLEMENTATIONS	10
IMPLEMENTATION OF CNN FOR OCEAN SURFACE TEMPERATURE PREDICTION	11
DESCRIPTION OF CNN ARCHITECTURE	11
TRAINING PROCESS AND HYPERPARAMETERS	12
RESULTS AND DISCUSSION	15
LIMITATIONS AND FUTURE DIRECTIONS	15
CONCLUSION	15
REFLECTIONS ON THE RESEARCH PROCESS	16
DECISION-MAKING AND RATIONALE	16
LEARNING AND SKILL DEVELOPMENT	16
SETBACKS AND ADAPTATIONS	17
BIBLIOGRAPHY	18
APPENDICES	21
APPENDIX A – DOWNLOADING DATA	21
APPENDIX B – COARSENING 0.01 DATA TO 0.05	24
APPENDIX C – PRODUCING PATCHES	26
APPENDIX D – EXPERIMENT PROCEDURE	28

Introduction

Background and Motivation

Oceans play a pivotal role in regulating the planet's climate and supporting various ecosystems. Accurate prediction of ocean surface temperature is crucial to understand these intricate processes. Traditional ocean surface temperature prediction models, while valuable, often face challenges in capturing the complex spatial and temporal patterns inherent in ocean dynamics (Fox-Kemper et al., 2019). This prompts the exploration of advanced computational techniques to enhance prediction accuracy.

In recent years, the field of computer science has witnessed remarkable progress in the realm of deep learning (Ahmad, 2019; Sinha & Abernathey, 2021), particularly Convolutional Neural Networks (CNN). These technologies have demonstrated remarkable capabilities in image recognition and restoration (Liu et al., 2018), data synthesis, and pattern extraction.

This essay seeks to explore the intersection of computer science and oceanography.

Research Question and Objectives

Our main goal revolves around the efficiency of Convolutional Neural Networks (CNN) in optimizing ocean surface temperature prediction models. The overarching research question guiding this exploration is:

"To what extent can Convolutional Neural Networks (CNN) enhance the accuracy and reliability of ocean surface temperature prediction models?"

Significance and Scope

The optimization of ocean surface temperature prediction models carries profound implications for a range of critical applications. Accurate and reliable predictions of ocean temperature are pivotal in understanding climate dynamics, marine ecosystems, and the intricate interplay between oceans and atmospheric systems. This extended essay aims to examine the application of Convolutional Neural Networks (CNN).

Accurate ocean surface temperature prediction is crucial for advancing climate models. Reliable predictions contribute to a deeper understanding of global climate patterns, aiding climate scientists and policymakers in making informed decisions. Precise temperature predictions enable effective ecosystem management. Marine organisms' distribution and behavior are profoundly influenced by temperature variations, making accurate predictions essential for the

conservation of marine life. Ocean temperatures significantly influence weather patterns. Improved prediction models can enhance the accuracy of weather forecasts, benefiting disaster preparedness and resource allocation. Oceanographers rely on temperature data for diverse research endeavors, from studying ocean currents to assessing the impacts of climate change on marine environments.

This extended essay focuses on optimizing ocean surface temperature prediction models specifically through the utilization of CNN techniques. A comprehensive analysis of techniques, exploring their underlying mechanisms and applicability in the domain of ocean surface temperature prediction. Rigorous experimentation involving the implementation and training of CNN models using real-world ocean surface temperature data. A meticulous comparison of the predictive performance of CNN to the models results produces with conventional prediction methods, quantifying enhancements in accuracy and reliability. An exploration of ethical considerations pertinent to data usage, privacy, and proper citation, ensuring responsible research practices.

Role of Computing in Ocean Surface Temperature Prediction

In the realm of oceanography, the integration of advanced computational techniques has revolutionized the accuracy and sophistication of surface temperature prediction models (Bolton & Zanna, 2019; Ducournau & Fablet, 2017). Traditionally, oceanographers relied on empirical approaches and simplified physical models. However, the dynamic and complex nature of ocean systems necessitates a more data-driven and intricate modeling strategy (Malde et al., 2019; Reichstein et al., 2019).

Modern computing enables the assimilation of vast datasets, encompassing variables such as ocean currents, atmospheric conditions, and historical temperature records. This data-driven approach empowers scientists to uncover nuanced patterns and correlations, refining prediction accuracy. The improvements in machine learning caused a paradigm shift. Algorithms, such as Convolutional Neural Networks (CNN) transcend conventional methods by autonomously identifying intricate relationships within multi-dimensional data. Computing accelerates the processing of massive datasets, facilitating high-resolution simulations that yield precise temperature forecasts. This precision contributes to a more profound comprehension of oceanic phenomena.

Convolutional Neural Networks excel at extracting hierarchical features from complex spatial data. In the context of ocean surface temperature prediction, CNNs can discern intricate temperature patterns influenced by diverse factors like ocean currents, topography, and solar radiation.

Relevance of CNN Techniques

In the contemporary landscape of ocean surface temperature prediction, the infusion of Convolutional Neural Networks (CNN) holds immense promise. These techniques, born from the realm of computer science, offer a transformative approach to refining the accuracy and predictive capabilities of oceanographic models.

CNN for Spatial Patterns:

1. **Feature Extraction:** Convolutional Neural Networks are adept at capturing intricate spatial patterns within datasets. In the context of oceanography, CNNs can discern temperature variations influenced by geographical features, such as ocean currents, coastal topography, and eddies.
2. **Hierarchical Learning:** CNN's hierarchical learning enables the model to identify features of varying scales, from minute temperature fluctuations to broader climate phenomena. This granularity contributes to a more nuanced understanding of temperature dynamics.

The integration of CNN techniques into ocean surface temperature prediction models marks a shift from traditional empirical approaches to data-driven, algorithmic methodologies. By harnessing the computational power of these techniques, oceanographers can unlock deeper insights into the intricate interplay of factors shaping surface temperature dynamics.

Methodology

Explanation of CNN Techniques

Convolutional Neural Networks (CNN) represent pivotal advancements in the realm of machine learning, each offering distinct yet synergistic capabilities.

Convolutional Neural Networks (CNN):

1. **Neural Architecture:** CNNs are inspired by the human visual cortex's structure, designed to effectively process grid-like data such as images. Layers of interconnected nodes, including convolutional, pooling, and fully connected layers, form its architecture.
2. **Feature Extraction:** CNNs excel in feature extraction, using convolutional layers to identify visual patterns within localized regions. These patterns progress hierarchically through the network, allowing the model to recognize complex features.

3. **Weight Sharing:** The concept of weight sharing reduces computational load by using the same set of weights across various regions, enhancing efficiency and enabling CNNs to generalize patterns.

Data Collection and Preprocessing

Accurate predictions of ocean surface temperatures necessitate high-quality and comprehensive datasets. In this section, we outline the systematic approach taken to collect and preprocess data, ensuring its suitability for training and evaluating our prediction models.

A multi-scale dataset (Chin et al., 2017) was used as both training and testing data, as it contains data in 0.01 degree and 0.25 degree spatial resolution. As these datasets have 25 times difference in terms of spatial resolution, which result in 625 times difference of the cell area, for initial experiment the 0.01 data is coarsened to 0.05.

Experimental Setup

To train the CNN we split all the data into the patches of the same areas, 200x200 pixels for high resolution data and 40x40 pixels for low resolution data, respectively. It will give same area coverage as their resolutions are 0.05 and 0.25 degrees per pixel.

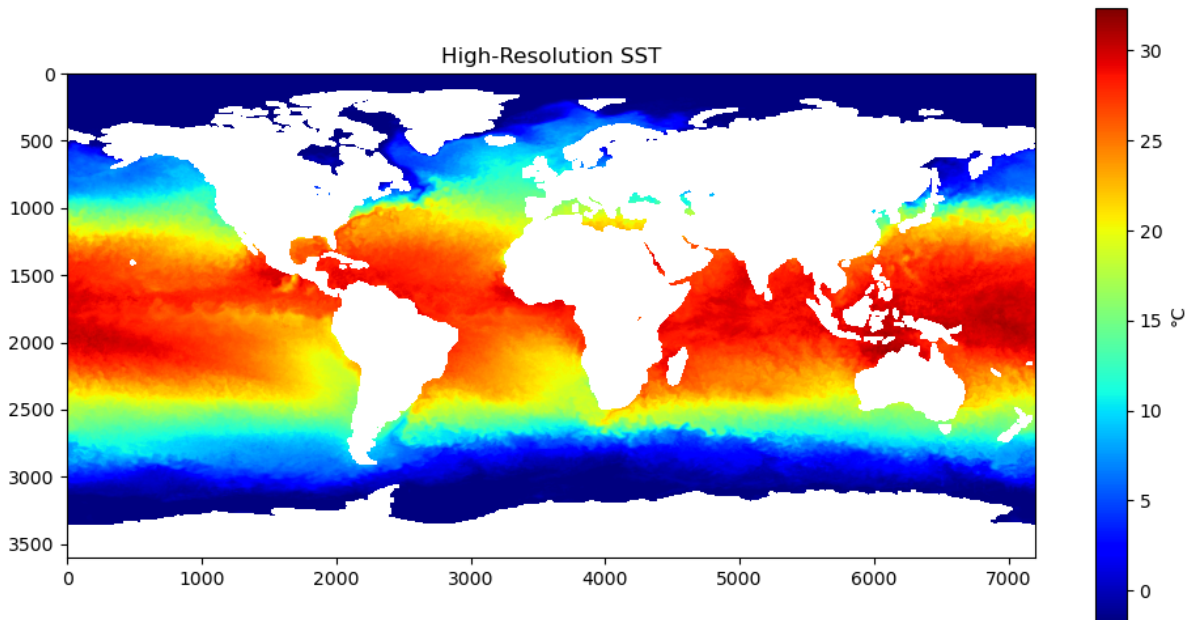


Figure 1 global sea surface temperature on 2003-12-01 in 0.05 resolution

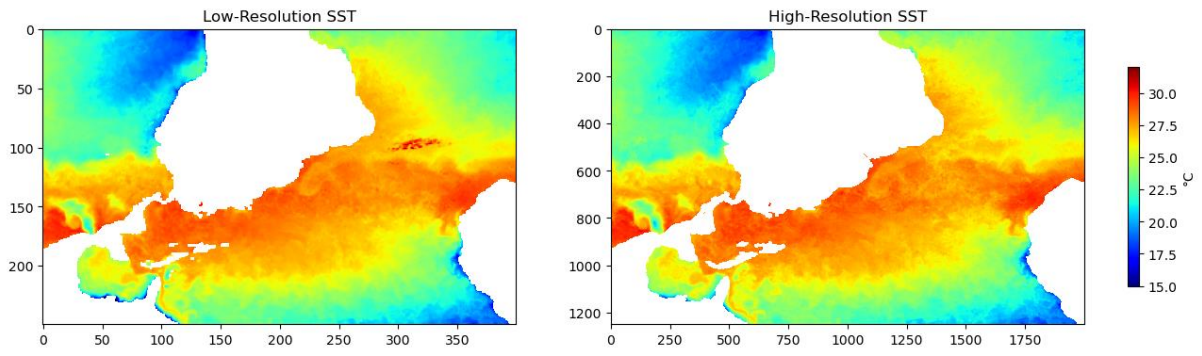


Figure 2 comparison of the same region of the low and high resolution data

The datasets come with areas representing land masses, which are not of interest in our study focused on ocean temperatures. These irrelevant areas were masked out, and the focus was kept strictly on the ocean surface temperatures. Additionally, patches from both resolutions were extracted. A patch is a smaller section of the image that we use for training the model. For the low-resolution images, patches of size 40x40 were extracted, and for high-resolution images, patches of size 200x200 of the same regions were extracted.

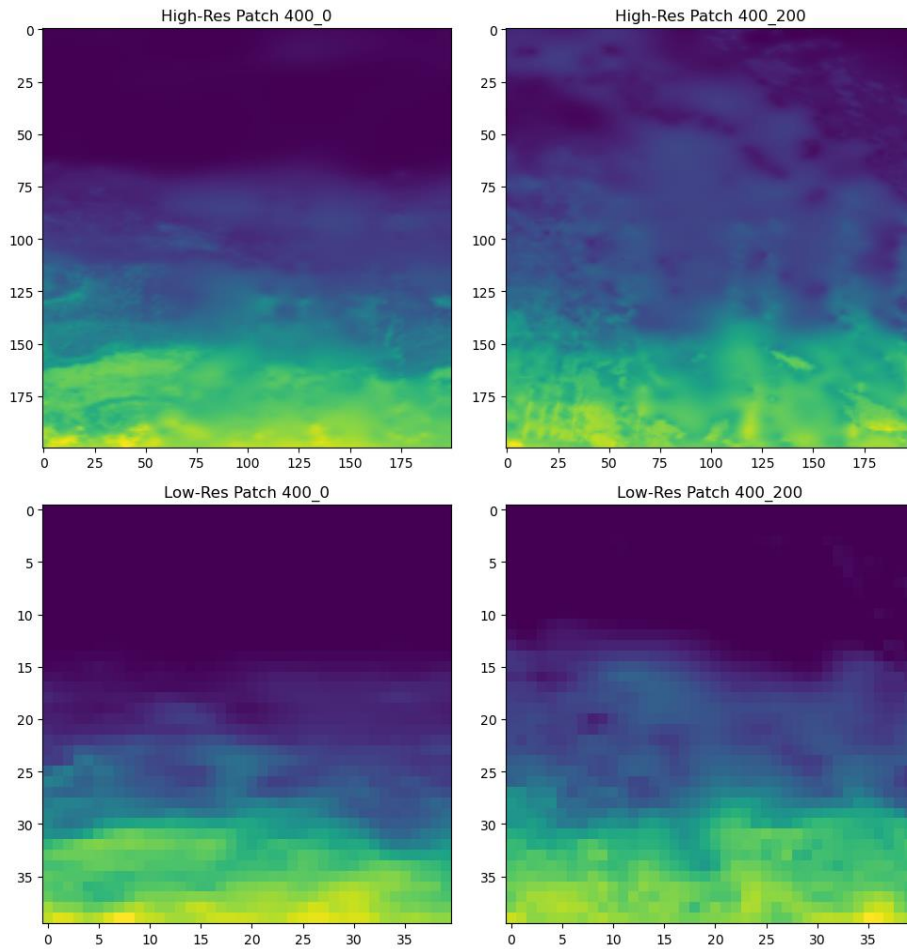


Figure 3 Comparison of the same-area patches in low(0.25) and high(0.05) resolution

First let's make a simple Convolutional Neural Network (CNN) for the super-resolution task. The CNN comprises multiple layers, including convolutional layers and upsampling layers. Convolutional layers are fundamental to CNNs, enabling the model to learn hierarchical features from the input images. The upsampling layers in the model allow the network to increase the resolution of the input image, and we use it to make the output image of the same resolution as the high-resolution ground truth.

During training, the model learned to map low-resolution patches to high-resolution patches. The objective was to minimize the difference between the model's predictions and the actual high-resolution patches. This process continued for several epochs, with each epoch representing a complete pass through the entire training dataset.

After training the model, we tested it to visualize and assess its performance. We presented low-resolution images to the model and observed the super-resolved outputs. Comparisons were made between the original high-resolution images and the super-resolved images produced by the model to evaluate the model's effectiveness in enhancing image resolution.

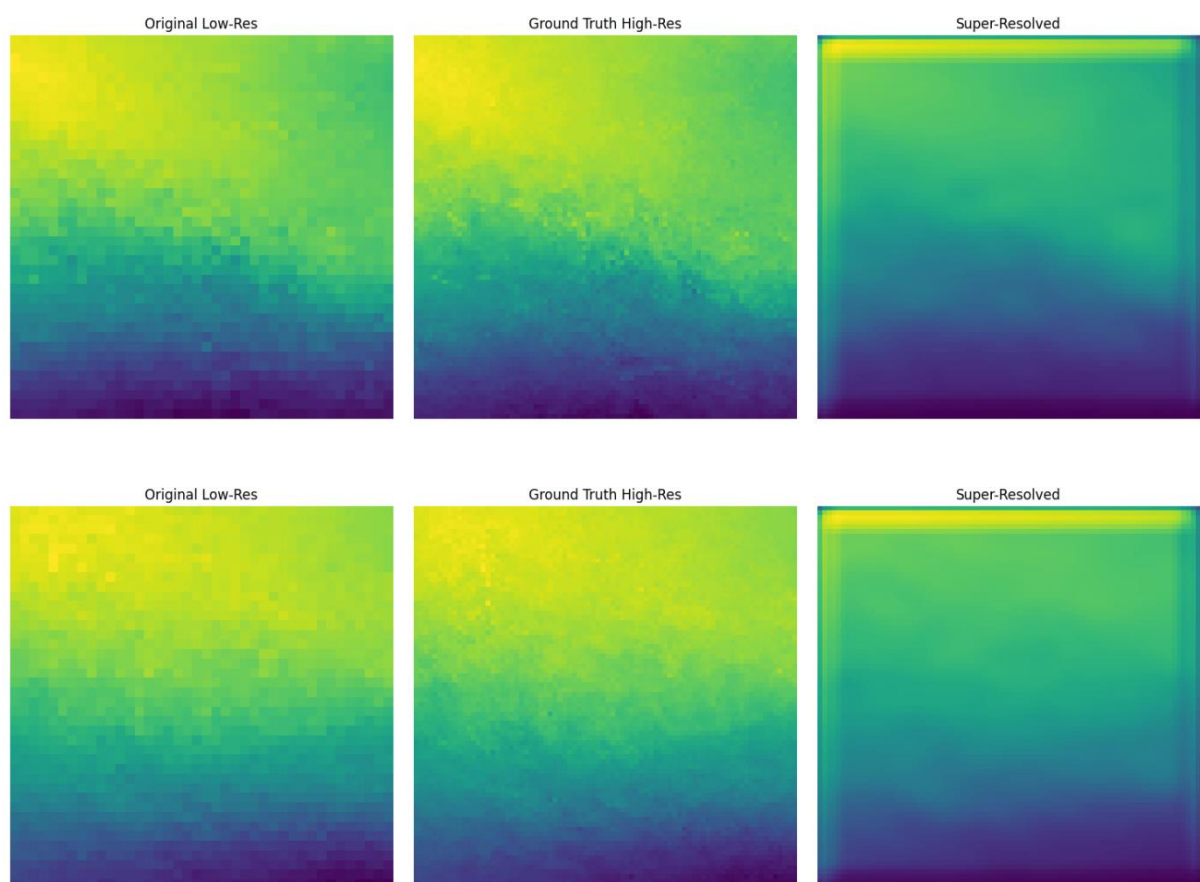


Figure 4 simple CNN output example. left to right: low resolution data patch, high resolution patch of the same region and model output

Literature Review

Current State of Ocean Surface Temperature Prediction Models

Predicting ocean surface temperature is a challenging task that has seen many improvements over time. Earlier methods depended on physical and statistical models, which were helpful but sometimes struggled with the ocean's unpredictable nature. These traditional models used past temperature data and fixed patterns to guess future temperatures. However, the ocean is always changing, and these models can't always keep up, leading to less accurate forecasts.

Understanding the ocean's temperature involves looking at a wide range of factors, from the quick breaking of waves to changes in the climate that take hundreds of years. It's about seeing the small details like the mixing of water that's only a few centimeters deep, as well as the huge patterns of how water moves around the entire planet (Gill, 1982). Creating models that can accurately show these vast and varied aspects of the ocean is a big challenge and shows just how complex and interesting the ocean really is. Numerical models are relatively new part of oceanography, with first 3-dimensional ocean circulation model proposed around half a century ago (Bryan & Cox, 1967).

Ocean and atmosphere modelling are quite similar because governing equations of fluid motions are the same. The differences origin in the different density and topography. First atmosphere circulation model appeared not a long before ocean one (Phillips, 1956). Some models use same code to model both atmosphere and ocean (Marshall et al., 1997).

One of the crucial model parameters is spatial resolution. It defines what physical phenomena the model can resolve. One type of significant scale-related processes is the mesoscale eddy. Resolving mesoscale eddies improves both lateral and vertical heat transport, has a huge impact on the model results (Griffies et al., 2015; Hewitt et al., 2016). On the other hand, increasing the spatial resolution requires non-linear increase in the computational resources needed for the model. As shown in (Hewitt et al., 2017), increasing spatial resolution twice increases computational cost by a factor of ten each time.

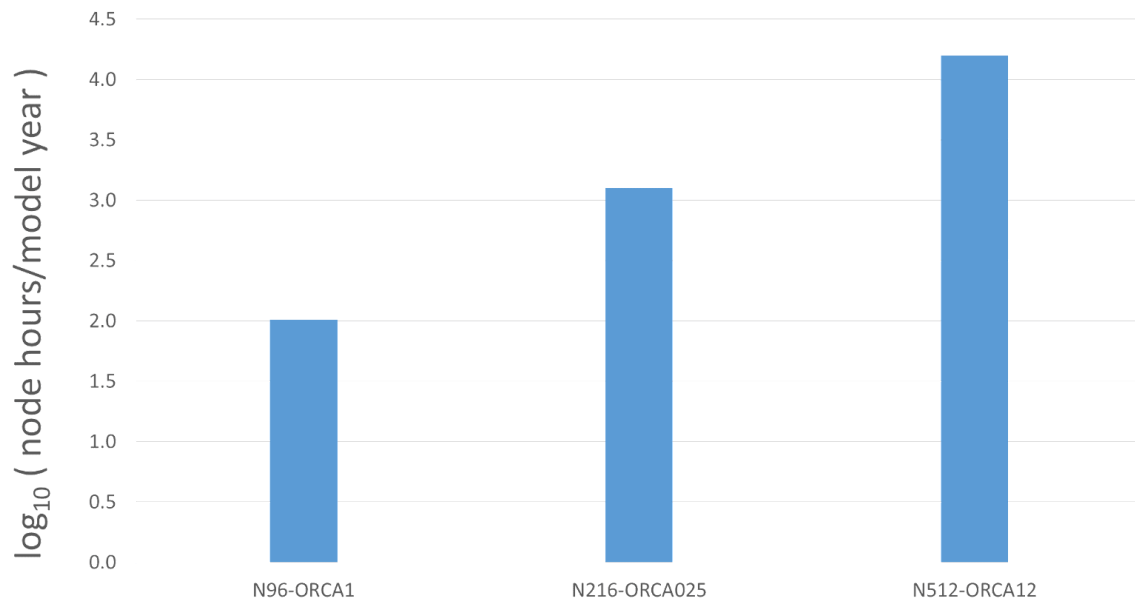


Figure 5 Example of an increase in computational cost (logarithmic scale) along with a spatial resolution

Evolution in resolution (1960-today)

In 1975 500km horizontal grid all over the globe was seen as a good model (Bryan et al., 1975), in 1989 (Zhang & Liang, 1989) made a $4^\circ \times 5^\circ$ global circulation ocean model. In the first local experiments resolutions were a magnitude of 1° . Bryan made an experiment on Indian Ocean horizontal domain with several resolutions: 4° , 2° , and 1° (Bryan, 1969).

AI implementations

Using Artificial Intelligence techniques in the ocean modelling has many implications: clustering water masses, analysing migration paths and more accurate representation of subgrid processes (Ahmad, 2019).

Implementation of CNN for Ocean Surface Temperature Prediction

Description of CNN Architecture

The design of the Convolutional Neural Network (CNN) architecture for super-resolution tasks is pivotal in determining the quality of the upscaled ocean surface temperature data. Our CNN is specifically tailored to handle the intricacies of two-dimensional temperature data represented in low-resolution image patches.

The initial layer of the CNN is the input layer, designed to ingest low-resolution temperature patches. These patches, typically of 40x40 pixels, represent the coarse data obtained from satellite measurements of ocean surface temperatures. Following the input layer is a cascade of convolutional layers. These layers are the workhorses of the CNN, each consisting of filters, also known as kernels, that convolve across the input image. As these filters move across the image, they perform element-wise multiplication and sum the results, producing feature maps. These feature maps form the core of the network's ability to detect patterns, edges, and textures within the temperature data. The depth and size of these filters are meticulously chosen to capture the wide spectrum of spatial features, from the subtlest gradients to the more pronounced thermal boundaries.

After each convolutional operation, we apply an activation function — typically the Rectified Linear Unit (ReLU). ReLU introduces non-linearity into the system, allowing the network to learn and represent more complex patterns that are not possible through linear combinations alone. To achieve super-resolution — the upscaling of image resolution — the architecture incorporates upsampling layers. These layers apply techniques such as nearest-neighbor or bilinear interpolation to enlarge the feature maps. Upsampling layers, placed closer to the output of the network fill in the finer details that were missing in the low-resolution input.

The climax of the CNN is the output layer, which is carefully designed to reconstruct the high-resolution temperature patch. This layer often uses a set of filters to combine the upsampled feature maps into a single image that matches the dimensions of the high-resolution target, typically 80x80 pixels. The output layer's filters are trained to refine the upscaled image, ensuring that the final output recaptures the rich details and texture present in the actual high-resolution temperature data.

The architecture of the CNN is a balanced interplay between depth (number of layers), width (number of filters), and resolution (size of feature maps), all orchestrated to optimize the network's performance in generating accurate, high-resolution predictions of ocean surface temperatures. Through iterative training and refinement, this CNN architecture aims to bridge the gap between

low-resolution measurements and high-fidelity temperature mapping necessary for advanced oceanographic research.

Training Process and Hyperparameters

The training process of the Convolutional Neural Network (CNN) is a critical phase where the model learns to upscale low-resolution ocean surface temperature patches into high-resolution predictions. This procedure involves several hyperparameters and steps, which are crucial for the success of the model.

Initialization of Hyperparameters:

Before the training commences, we define a set of hyperparameters that govern the learning process:

- **Learning Rate:** Typically starting from a value like 0.001, the learning rate is of utmost importance as it determines the magnitude of the updates to the model's weights during training. A rate too high might overshoot the optimal solution, while a rate too low may result in excessively long training times or getting stuck in local minima.
- **Batch Size:** This parameter specifies the number of training examples utilized in one iteration. For instance, a batch size of 32 means that 32 low-resolution patches are fed into the network at once, and the average loss is computed before updating the model weights.
- **Epochs:** An epoch represents one full cycle through the entire training dataset. The number of epochs is set based on the size of the dataset and the complexity of the task, often ranging from tens to hundreds, to allow the network sufficient time to learn the underlying patterns.
- **Optimizer:** Algorithms like Adam are used to optimize the network's weights. Adam is favored for its adaptive learning rate capabilities.
- **Loss Function:** Mean Squared Error (MSE) is the typical choice for regression tasks like super-resolution. It measures the average squared difference between the estimated values and the actual value.

Training Procedure:

The actual training process follows these steps:

1. **Forward Pass:** The low-resolution patches are input into the CNN, which processes them through its layers to output the predicted high-resolution patches.
2. **Loss Calculation:** The MSE loss function calculates the discrepancy between the network's predictions and the ground truth high-resolution patches.
3. **Backpropagation:** This mechanism calculates the gradient of the loss function with respect to each weight in the network by propagating the error backward through the model.

4. **Weight Update:** The optimizer uses these gradients to update the weights. If using Adam, it adapts the learning rate for each weight individually.
5. **Learning Rate Scheduler:** To further refine the training, a scheduler may be used to decrease the learning rate at certain epochs, helping the model to converge more smoothly towards the optimal weights.

We monitor the model's performance on a testing set, which is not used for training but to evaluate how well the model is learning and generalizing. If the validation loss starts to increase, it may indicate that the model is beginning to overfit the training data.

Finally, the model is trained across the defined number of epochs, with periodic evaluation to track progress and adjust as needed. This iterative process continues until the model's performance on the testing set plateaus or begins to degrade, at which point the training can be halted, and the best-performing model is saved for further testing and analysis.

Results:

Epoch 10

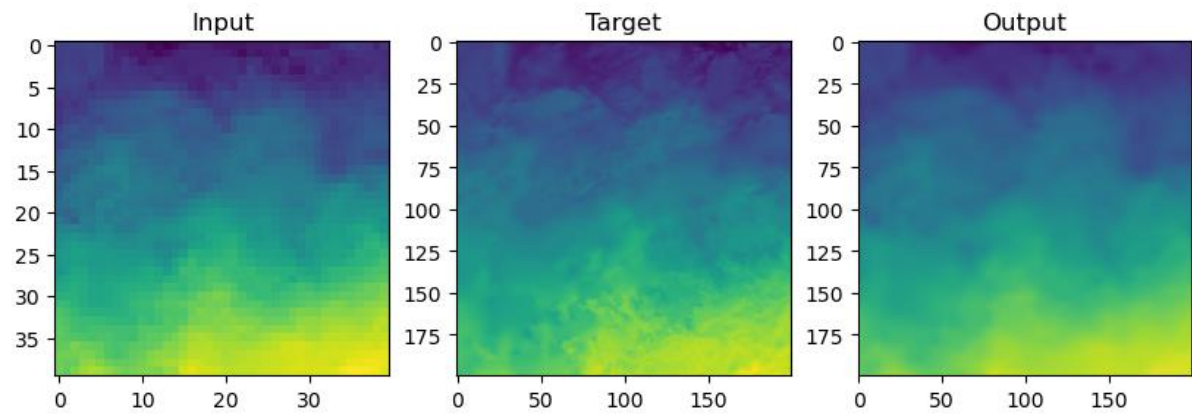


Figure 6 Result after 10 epochs of architecture 1.

Epoch 1

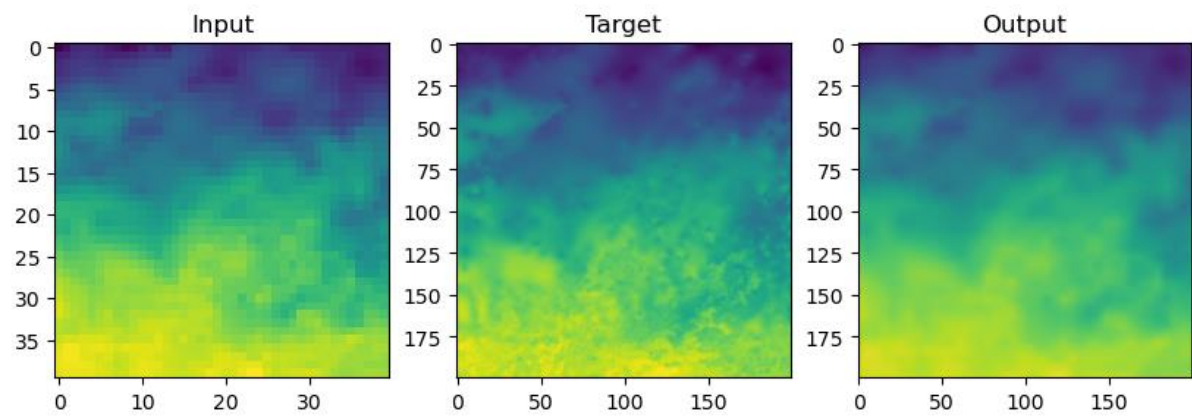


Figure 7 Model after the first epoch.

Results and Discussion

The goal to develop a super-resolution model has yielded promising results, with the final outputs appearing visually correct. Yet, upon closer examination, several issues became evident. Blurriness, a common problem in upscaling tasks, was present, resulting in a loss of detail that is essential for high-resolution images to be considered truly high quality. Additionally, edge artifacts – undesired distortions around image edges – were observed. These artifacts are particularly problematic as they disrupt the natural lines and shapes within an image, leading to a less realistic appearance. Both blurriness and edge artifacts significantly affect the visual appeal and clarity of the upscaled images. For practical applications, such as medical imaging or satellite imagery where detail is paramount, these challenges undermine the utility of the super-resolution model.

Limitations and Future Directions

To quantify the model's performance, interpolation methods such as nearest-neighbor, bilinear, and bicubic upsampling may be used as benchmarks. These traditional methods, while simpler, provide a baseline for understanding the improvements and shortcomings of the neural network approach.

The limitations encountered suggest that the current model may not possess the depth or complexity necessary to fully overcome the intricacies of upscaling. Blurriness could be addressed by deeper architectural designs that can encapsulate more contextual information at various scales. Similarly, the occurrence of edge artifacts calls for improved edge detection and preservation techniques within the network.

Future research should explore the integration of more advanced loss functions that better correspond to human visual perception, which could offer a more nuanced approach to training the model. Additionally, the use of generative adversarial networks (GANs) could be investigated to produce sharper images, as the adversarial process can often generate images that are more crisp and closer to the high-resolution target.

Conclusion

The iterative process of developing a super-resolution model has been insightful, providing a deeper understanding of the challenges inherent to image upscaling. The project highlighted the need for models that not only increase the pixel count but do so in a way that enhances the perceptual quality of images. The journey underscored the importance of continual testing, evaluation, and refinement in the field of neural networks and artificial intelligence. As the technology advances, the aspiration to achieve super-resolution images that are indistinguishable

from their high-resolution counterparts remains a compelling goal, driving forward the frontier of image processing research.

Reflections on the Research Process

Decision-Making and Rationale

The inception of the project was rooted in the quest to enhance the resolution of oceanic imagery, a task vital for various applications including climate monitoring and marine life conservation. The decision to employ Convolutional Neural Networks (CNNs) was informed by their proven efficacy in image analysis and pattern recognition. A systematic approach was adopted to gather and preprocess high-resolution and low-resolution oceanic images, laying the groundwork for a robust dataset.

The rationale behind selecting a super-resolution model was to improve upon traditional interpolation methods, which often introduce blurriness and fail to reconstruct high-frequency details. Through iterative trials, a model architecture incorporating residual blocks and skip connections was chosen for its ability to mitigate the degradation problem and preserve edge information—a crucial factor in maintaining the fidelity of ocean topography.

Learning and Skill Development

The project was instrumental in fostering a comprehensive understanding of neural network architecture, specifically CNNs. Through hands-on experience, I developed proficiency in data acquisition and preprocessing—an essential skill set for feeding neural networks with high-quality data.

The practical implementation of the model underscored the importance of model architecture decisions. Experimenting with different layers and connections revealed the significant impact of such choices on the model's output. The process of training the network and observing the impact of various loss functions on image quality facilitated a deeper appreciation for the nuanced trade-offs between mathematical optimization and perceptual relevance.

Comparing the quality of the model's output with baseline methods such as interpolation not only provided a benchmark for progress but also offered insights into the limitations of current techniques. This comparative analysis is crucial for evaluating the model's effectiveness and identifying areas for future improvement.

The journey through the project emphasized the iterative nature of machine learning and the critical thinking involved in refining a model. Each challenge encountered, such as dealing with

blurriness and edge artifacts, served as a learning opportunity, guiding subsequent adjustments to the model and training process.

Setbacks and adaptations

The journey of developing a super-resolution model was not without its challenges. Early iterations of the model produced images that, while higher in resolution, lacked the sharpness and clarity of the original high-resolution samples. The initial use of Mean Squared Error (MSE) as a loss function led to a mathematical optimization that did not fully account for the spatial features.

1. In response, the model was restructured to incorporate residual blocks and skip connections, drawing inspiration from the success of architectures like ResNet. These adaptations aimed to preserve and enhance detail through each layer of the neural network, combating the blurriness that often accompanies image upscaling.

Bibliography

1. Ahmad, H. (2019). Machine learning applications in oceanography. *Aquatic Research*, 161–169. <https://doi.org/10.3153/ar19014>
2. Bolton, T., & Zanna, L. (2019). Applications of Deep Learning to Ocean Data Inference and Subgrid Parameterization. *Journal of Advances in Modeling Earth Systems*, 11(1), 376–399. <https://doi.org/10.1029/2018MS001472>
3. Bryan, K. (1969). A numerical method for the study of the circulation of the world ocean. *Journal of Computational Physics*, 4(3), 347–376.
4. Bryan, K., & Cox, M. D. (1967). A numerical investigation of the oceanic general circulation. *Tellus*, 19(1), 54–80. <https://doi.org/10.1111/j.2153-3490.1967.tb01459.x>
5. Bryan, K., Manabe, S., & Pacanowski, R. C. (1975). A Global Ocean-Atmosphere Climate Model. Part II. The Oceanic Circulation. *Journal of Physical Oceanography*, 5(1), 30–46. [https://doi.org/10.1175/1520-0485\(1975\)005<0030:agoacm>2.0.co;2](https://doi.org/10.1175/1520-0485(1975)005<0030:agoacm>2.0.co;2)
6. Chin, T. M., Vazquez-Cuervo, J., & Armstrong, E. M. (2017). A multi-scale high-resolution analysis of global sea surface temperature. *Remote Sensing of Environment*, 200, 154–169. <https://doi.org/10.1016/J.RSE.2017.07.029>
7. De Bézenac, E., Pajot, A., & Gallinari, P. (2018). Deep learning for physical processes: Incorporating prior scientific knowledge. 6th International Conference on Learning Representations, ICLR 2018 - Conference Track Proceedings.
8. Déchelle-Marquet, M., Levy, M., Gallinari, P., Crepon, M., Thiria, S.. (2020). Deep neural networks to downscale ocean climate models. EGUGA, 20177. <https://ui.adsabs.harvard.edu/abs/2020EGUGA..2220177D/abstract>
9. Ducournau, A., & Fablet, R. (2017, February 28). Deep learning for ocean remote sensing: An application of convolutional neural networks for super-resolution on satellite-derived SST data. 2016 9th IAPR Workshop on Pattern Recognition in Remote Sensing, PRRS 2016. <https://doi.org/10.1109/PRRS.2016.7867019>
10. Fablet, R., Ouala, S., & Herzet, C. (2017). Bilinear residual Neural Network for the identification and forecasting of dynamical systems. <http://arxiv.org/abs/1712.07003>
11. Fox-Kemper et al. (2019). Challenges and prospects in ocean circulation models. *Frontiers in Marine Science*, 6(FEB), 65. <https://doi.org/10.3389/fmars.2019.00065>
12. Gill, A. E. (1982). *Atmosphere–Ocean Dynamics*. International Geophysics Series.
13. Griffies, S. M., Winton, M., Anderson, W. G., Benson, R., Delworth, T. L., Dufour, C. O., Dunne, J. P., Goddard, P., Morrison, A. K., Rosati, A., Wittenberg, A. T., Yin, J., & Zhang, R. (2015). Impacts on ocean heat from transient mesoscale eddies in a hierarchy of climate models. *Journal of Climate*, 28(3), 952–977. <https://doi.org/10.1175/JCLI-D-14-00353.1>

14. Hewitt, H. T., Bell, M. J., Chassignet, E. P., Czaja, A., Ferreira, D., Griffies, S. M., Hyder, P., Mcclean, J. L., New, A. L., & Roberts, M. J. (2017). Will high-resolution global ocean models benefit coupled predictions on short-range to climate timescales? *Ocean Modelling*, 120, 120–136. <https://doi.org/10.1016/j.ocemod.2017.11.002>
15. Hewitt, H. T. et al. (2016). The impact of resolving the Rossby radius at mid-latitudes in the ocean: Results from a high-resolution version of the Met Office GC2 coupled model. *Geoscientific Model Development*, 9(10), 3655–3670. <https://doi.org/10.5194/gmd-9-3655-2016>
16. Liu, P., Zhang, H., Zhang, K., Lin, L., & Zuo, W. (2018). Multi-level Wavelet-CNN for Image Restoration. *IEEE Computer Society Conference on Computer Vision and Pattern Recognition Workshops*, 2018-June, 886–895. <https://arxiv.org/abs/1805.07071v2>
17. Malde, K., Handegard, N. O., Eikvil, L., & Salberg, A.-B. (2019). Machine Intelligence and the data-driven future of science. *ICES Journal of Marine Science*. <https://doi.org/10.1093/icesjms/fsz057>
18. Marshall, J., Adcroft, A., Hill, C., Perelman, L., & Heisey, C. (1997). A finite-volume, incompressible Navier Stokes model for studies of the ocean on parallel computers. *Journal of Geophysical Research: Oceans*, 102(C3), 5753–5766. <https://doi.org/10.1029/96JC02775>
19. Phillips, N. A. (1956). The general circulation of the atmosphere: A numerical experiment. *Quarterly Journal of the Royal Meteorological Society*, 82(352), 123–164. <https://doi.org/10.1002/qj.49708235202>
20. Reichstein, M., Camps-Valls, G., Stevens, B., Jung, M., Denzler, J., Carvalhais, N., & Prabhat. (2019). Deep learning and process understanding for data-driven Earth system science. *Nature*, 566(7743). <https://doi.org/10.1038/s41586-019-0912-1>
21. Rodrigues, E. R., Oliveira, I., Cunha, R., & Netto, M. (2018). DeepDownscale: A deep learning strategy for high-resolution weather forecast. *Proceedings - IEEE 14th International Conference on EScience, e-Science 2018*. <https://doi.org/10.1109/eScience.2018.00130>
22. Sinha, A., & Abernathey, R. (2021). Estimating Ocean Surface Currents With Machine Learning. *Frontiers in Marine Science*, 0, 612. <https://doi.org/10.3389/FMARS.2021.672477>
23. Zhang, X., & Liang, X. (1989). A numerical world ocean general circulation model. *Advances in Atmospheric Sciences*, 6(1), 44–61. <https://doi.org/10.1007/BF02656917>

Appendices

Appendix A - Downloading data

```
import requests
```

```
# Importing the 'requests' library for making HTTP requests in Python.
```

```
from datetime import datetime, timedelta
```

```
# Importing 'datetime' and 'timedelta' from the datetime module for working with dates and time.
```

```
class SessionWithHeaderRedirection(requests.Session):
```

```
# Defining a subclass of 'requests.Session' to handle specific redirection behavior.
```

```
AUTH_HOST = 'urs.earthdata.nasa.gov'
```

```
# Setting a constant for the authentication host.
```

```
def __init__(self, username, password):
```

```
    super().__init__()
```

```
# Initializing the superclass (requests.Session).
```

```
    self.auth = (username, password)
```

```
# Setting the username and password for authentication.
```

```
def rebuild_auth(self, prepared_request, response):
```

```
# Overriding the 'rebuild_auth' method to handle the 'Authorization' header on redirection.
```

```
    headers = prepared_request.headers
```

```
# Getting headers from the prepared request.
```

```
    url = prepared_request.url
```

```
# Getting the URL from the prepared request.
```

```
    if 'Authorization' in headers:
```

```
# Checking if the 'Authorization' header is present.
```

```
    original_parsed = requests.utils.urlparse(response.request.url)
```

```
    redirect_parsed = requests.utils.urlparse(url)
```

```
# Parsing the original and redirect URLs.
```

```
    if (original_parsed.hostname != redirect_parsed.hostname) and \
```

```
        redirect_parsed.hostname != self.AUTH_HOST and \
```

```
        original_parsed.hostname != self.AUTH_HOST:
```

```
        # Checking if the hostname has changed during redirection, excluding the
AUTH_HOST.
```

```
    del headers['Authorization']
```

```
    # Deleting the 'Authorization' header.
```

```
    return
```

```
    # Returning from the function.
```

```
username = input("username")
```

```
password = input("password")
```

```
# Prompting the user to enter their username and password.
```

```
session = SessionWithHeaderRedirection(username, password)
```

```
# Creating an instance of the custom session class with the provided credentials.
```

```
def download_file(url, filename):
```

```
    # Defining a function to download a file from a URL.
```

```
    with session.get(url, stream=True) as response:
```

```
        # Making a GET request with streaming enabled.
```

```
        response.raise_for_status()
```

```
        # Raising an exception for HTTP error responses.
```

```
        with open(filename, 'wb') as file:
```

```
            # Opening a file to write binary data.
```

```
            for chunk in response.iter_content(chunk_size=8192):
```

```
                # Reading the response in chunks.
```

```
                file.write(chunk)
```

```
                # Writing each chunk to the file.
```

```
date = datetime(2003, 12, 9)
```

```
end_date = datetime(2004, 1, 1)
```

```
# Setting the start and end dates for data download.
```

```
date_fmt = "%Y%m%d"
```

```
# Defining the date format for the filenames.
```

```
while date < end_date:
```

```
    # Looping through dates from start date to end date.
```

```

date_str = date.strftime(date_fmt)
# Formatting the date as a string.

hr_url = f"https://archive.podaac.earthdata.nasa.gov/podaac-ops-cumulus-protected/MUR-
JPL-L4-GLOB-v4.1/{date_str}090000-JPL-L4_GHRSST-SSTfnd-MUR-GLOB-v02.0-fv04.1.nc"
lr_url = f"https://archive.podaac.earthdata.nasa.gov/podaac-ops-cumulus-protected/MUR25-
JPL-L4-GLOB-v04.2/{date_str}090000-JPL-L4_GHRSST-SSTfnd-MUR25-GLOB-v02.0-
fv04.2.nc"
# Constructing URLs for high-resolution and low-resolution data.

hr_filename = f"E:/data/ocean_data/MUR/v02/{date_str}.nc"
lr_filename = f"E:/data/ocean_data/MUR/v01/{date_str}.nc"
# Setting filenames for the downloaded data.

print(f"Downloading {hr_url} to {hr_filename}")
download_file(hr_url, hr_filename)
# Downloading the high-resolution data.
print(f"Downloading {lr_url} to {lr_filename}")
download_file(lr_url, lr_filename)
# Downloading the low-resolution data.

date += timedelta(days=1)
# Incrementing the date by one day.

```

Appendix B - Coarsening 0.01 data to 0.05

```
import pandas as pd
from tqdm import tqdm
import os
import xarray as xr

# Importing required libraries: xarray for handling multi-dimensional arrays, numpy for numerical
# operations,
# matplotlib.pyplot for plotting, pandas for data manipulation, tqdm for progress bars, and os for
# interacting
# with the file system.

def save_coarsened(ds_high_res, date_str):
    # Defining a function to coarsen and save high resolution data.
    high_res_coarsened = ds_high_res.isel(time=0, lat=slice(None, None, 5), lon=slice(None,
None, 5))
    # Coarsening the dataset by selecting every 5th value along the latitude and longitude
    # dimensions.
    # .isel() - pandas index selection

    os.makedirs(f'coarsened/high_res/{date_str}', exist_ok=True)
    # Creating the directory if it doesn't exist, to save the coarsened data.

    high_res_coarsened.to_netcdf(f'coarsened/high_res/{date_str}/coarsened.nc')
    # Saving the coarsened data as a netCDF file in the specified directory.

def process_files(start_date, end_date):
    # Defining a function to process files within a date range.
    for date in tqdm(pd.date_range(start_date, end_date)):
        # Iterating over each date in the specified range.
        date_str = date.strftime('%Y%m%d')
        # Formatting the date as a string in 'YYYYMMDD' format.
        with xr.open_dataset(f"E:\\data\\ocean_data\\MUR\\v02\\{date_str}090000-JPL-
L4_GHRSSST-SSTfnd-MUR-GLOB-v02.0-fv04.1.nc") \
            as ds_high_res:
            # Opening the dataset file for the specified date. The file path seems to be hard-coded
```



```
save_coarsened(ds_high_res, date_str)
```

```
# Calling the function to coarsen and save the data for the current date.
```

```
# Usage:
```

```
start_date = '2003-12-01'
```

```
end_date = '2003-12-01'
```

```
process_files(start_date, end_date)
```

```
# Example usage of the process_files function to process data for a specific date range.
```

Appendix C - Producing patches

```
import matplotlib.patches as patches
```

Importing the 'patches' module from matplotlib for creating shapes like rectangles on plots.

```
import xarray as xr
```

Importing 'xarray', which is a library for handling multi-dimensional arrays with labeled dimensions.

```
import numpy as np
```

Importing 'numpy' for numerical operations on arrays.

```
import matplotlib.pyplot as plt
```

Importing 'pyplot' from 'matplotlib' for creating plots.

```
def save_patch_locations(ds_high_res, date_str):
```

Defining a function to save patch locations from high resolution data.

```
    high_res_data = ds_high_res['analysed_sst'].values
```

Extracting the sea surface temperature values from the dataset.

.values extracts only necessary values from ds_high_res and returns the data as a NumPy array.

Create a figure for the global map with patch locations

```
    fig, ax = plt.subplots(figsize=(10, 5))
```

Creating a matplotlib figure and axes.

```
    ax.imshow(high_res_data, cmap='viridis', origin='lower')
```

Displaying the high resolution data as an image with a specific colormap.

(origin = lower ensures correct vertical orientation)

```
    p_count = 0
```

Initializing a counter for the number of patches.

```
    for i in range(0, high_res_data.shape[0] - 200, 200):
```

```
        for j in range(0, high_res_data.shape[1] - 200, 200):
```

Iterating over the high resolution data in 200x200 patches.

```
            high_res_patch = high_res_data[i:i + 200, j:j + 200]
```

Extracting each patch.

```
            if np.isnan(high_res_patch).sum() == 0:
```

Check if the patch does not contain NaN values.

```

    # Draw a rectangle around the patch
    rect = patches.Rectangle((j, i), 200, 200, linewidth=1, edgecolor='r', facecolor='none')
    # Creating a rectangle patch.
    ax.add_patch(rect)
    # Adding the rectangle patch to the axes.

    p_count += 1
    # Incrementing the patch counter.


ax.set_title('Patch Locations')
# Setting the title of the plot.
# Save the global map with patch locations
plt.savefig('patches/global_map_with_patches_{date_str}.png')
# Saving the figure to a file.
plt.close(fig)
# Closing the figure to free up memory.


def process_single_date(date_str):
    # Defining a function to process data for a single date.
    with xr.open_dataset(r"D:\coarsened\high_res\date_str\coarsened.nc") as ds_high_res:
        # Opening the dataset for the given date string using xarray.
        save_patch_locations(ds_high_res, date_str)
        # Calling the function to save patch locations.


# Usage:
process_single_date('20231201')
# Example usage of the function to process data for a specific date.

```

Appendix E - Experiment procedure: CNN structure, training and testing

```
%matplotlib inline
```

```
# This line is a magic function for IPython that sets the backend of matplotlib to the 'inline' backend.
```

```
# With this backend, the output of plotting commands is displayed inline within Jupyter notebooks.
```

```
import torch
```

```
# Importing the PyTorch library, which is a popular library for building neural networks.
```

```
import torch.nn as nn
```

```
# Importing the neural network module from PyTorch, which contains classes to build different types of layers.
```

```
import torch.optim as optim
```

```
# Importing the optimization module from PyTorch, which includes various optimization algorithms like Adam, SGD, etc.
```

```
from torch.utils.data import DataLoader, TensorDataset
```

```
# Importing DataLoader and TensorDataset from PyTorch utils to handle and batch data for training.
```

```
from PIL import Image
```

```
# Importing the Python Imaging Library (PIL), which provides support for opening, manipulating, and saving many different image file formats.
```

```
import matplotlib.pyplot as plt
```

```
# Importing the pyplot interface of Matplotlib, which provides a MATLAB-like plotting framework.
```

```
import glob
```

```
# Importing the glob module to retrieve files/pathnames matching a specified pattern.
```

```
import numpy as np
```

```
# Importing NumPy, a fundamental package for scientific computing with Python.
```

```
from sklearn.model_selection import train_test_split
```

Importing train_test_split function from Scikit-learn for splitting data arrays into two subsets: for training data and for testing data.

```
from tqdm import tqdm
```

Importing tqdm, which is a library for adding progress bars to loops.

```
import torch.nn.functional as F
```

Importing the functional API from PyTorch, which contains functions like activation functions, loss functions etc.

Function to process a single image path

```
def process_path(file_path):
```

```
    img = Image.open(file_path)
```

Open an image file.

```
    img = img.convert('RGB') # Ensure 3 channels
```

Convert the image to RGB mode (3 channels).

```
    return np.array(img)
```

Return the image as a numpy array.

Load file paths

```
print("Loading file paths...")
```

```
low_res_paths = glob.glob("C:/data/patches/low_res/*/*.png")
```

Get a list of file paths for low resolution images.

```
high_res_paths = glob.glob("C:/data/patches/high_res/*/*.png")
```

Get a list of file paths for high resolution images.

Process images

```
print("Processing images...")
```

```
low_res_imgs = [process_path(p) for p in low_res_paths]
```

Process and store low resolution images.

```
high_res_imgs = [process_path(p) for p in high_res_paths]
```

Process and store high resolution images.

```
print('total low_res_imgs: ', len(low_res_imgs))
```

```
print('total high_res_imgs: ', len(high_res_imgs))
```

Print the total number of low and high resolution images.

```

# Convert to torch tensors and normalize to [0, 1]
print("Converting to torch tensors...")
x_data = torch.tensor([img.transpose((2, 0, 1)) for img in low_res_imgs], dtype=torch.float32) /
255.0
# Convert low resolution images to torch tensors and normalize pixel values.
y_data = torch.tensor([img.transpose((2, 0, 1)) for img in high_res_imgs], dtype=torch.float32) /
255.0
# Convert high resolution images to torch tensors and normalize pixel values.

# Split data
print("Splitting data...")
x_train, x_test, y_train, y_test = train_test_split(x_data, y_data, test_size=0.1, random_state=42)
# Split the data into training and testing sets.
train_dataset = TensorDataset(x_train, y_train)
# Create a training dataset. (encapsulate tensors into a dataset format)
test_dataset = TensorDataset(x_test, y_test)
# Create a testing dataset. (encapsulate tensors into a dataset format)
train_loader = DataLoader(train_dataset, batch_size=32, shuffle=True)
# Create a DataLoader for the training dataset.
test_loader = DataLoader(test_dataset, batch_size=32, shuffle=False)
# Create a DataLoader for the testing dataset.

# Create model
class SuperResolutionModel(nn.Module):
    # Define a class for the Super Resolution Model (inherits from nn.module).
    def __init__(self):
        super(SuperResolutionModel, self).__init__()
        # Initialize the model (layers).
        self.initial_upsample = nn.Upsample(scale_factor=5, mode='bilinear', align_corners=True)
        # Define an initial upsample layer.
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        # Define the first convolution layer. (input, output (amount of filters))
        self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        # Define the second convolution layer.
        self.conv3 = nn.Conv2d(128, 64, kernel_size=3, padding=1)
        # Define the third convolution layer.
        self.conv4 = nn.Conv2d(64, 3, kernel_size=3, padding=1)
        # Define the fourth convolution layer.

```

```

def forward(self, x):
    # Define the forward pass of the model.
    x_initial_upsampled = self.initial_upsample(x)
    # Apply the initial upsample layer.
    x1 = F.relu(self.conv1(x_initial_upsampled))
    # Apply the first convolution layer and ReLU activation.
    x2 = F.relu(self.conv2(x1))
    # Apply the second convolution layer and ReLU activation.
    x3 = F.relu(self.conv3(x2) + x1) # skip connection
    # Apply the third convolution layer with a skip connection and ReLU activation.
    x4 = self.conv4(x3) + x_initial_upsampled # skip connection
    # Apply the fourth convolution layer with a skip connection.
    return x4
    # Return the output.

```

```

def visualize_results(model, x, y, epoch):
    # Define a function to visualize results.
    with torch.no_grad():
        # Turn off gradients for evaluation.
        output = model(x)
        # Get the model output.
        output_img = output.squeeze().permute(1, 2, 0).numpy()
        # Process the output image for visualization.
        input_img = x.squeeze().permute(1, 2, 0).numpy()
        # Process the input image for visualization.
        target_img = y.squeeze().permute(1, 2, 0).numpy()
        # Process the target image for visualization.
        plt.figure(figsize=(10, 6))
        # Create a figure.
        plt.subplot(1, 3, 1)
        plt.title('Input')
        plt.imshow(input_img)
        # Show the input image.
        plt.subplot(1, 3, 2)
        plt.title('Target')
        plt.imshow(target_img)

```

```

    # Show the target image.
    plt.subplot(1, 3, 3)
    plt.title('Output')
    plt.imshow(output_img)
    # Show the output image.
    plt.suptitle(f'Epoch {epoch}')
    plt.show()
    # Show the figure.

model = SuperResolutionModel()
# Instantiate the model.
print(model)
# Print the model architecture.
criterion = nn.MSELoss()
# Define the loss function (Mean Squared Error Loss).
optimizer = optim.Adam(model.parameters())
# Define the optimizer (Adam).

# Train model
print("Training model...")
for epoch in range(10):
    # Train for 10 epochs.
    model.train()
    # Set the model to training mode.
    progress_bar = tqdm(train_loader, desc=f'Epoch {epoch + 1}')
    # Create a progress bar for the training loop.
    for x_batch, y_batch in progress_bar:
        # Iterate over batches.
        optimizer.zero_grad()
        # Zero the gradient buffers.
        outputs = model(x_batch)
        # Get model outputs for the current batch.
        loss = criterion(outputs, y_batch)
        # Compute the loss.
        loss.backward()
        # Backpropagate the error.
        optimizer.step()

```



```
# Update the weights.
progress_bar.set_postfix({'loss': loss.item()})
# Update the progress bar with the current loss.

# Save model checkpoint after each epoch
torch.save(model.state_dict(), f'model_checkpoint_epoch_{epoch + 1}.pth')
# Save the current state of the model.

# Visualize intermediate results
model.eval()
# Set the model to evaluation mode.
visualize_results(model, x_test[0].unsqueeze(0), y_test[0].unsqueeze(0), epoch + 1)
# Visualize the results after each epoch.
```